

Documentação do Projeto: PRISMA

🕒 Last edited time	@26 de setembro de 2025 13:12
📁 Projeto	 <u>Prisma IA</u>

1. Visão Geral do Projeto

O PRISMA TAI é uma plataforma web projetada para a criação, aplicação e análise de Testes de Associação Implícita (IAT) e pesquisas cognitivas, com foco no comportamento do consumidor. O sistema permite que pesquisadores desenvolvam e distribuam questionários personalizados, utilizando templates, para coletar e analisar dados de forma eficiente. A plataforma é projetada para ser acessível tanto em dispositivos desktop quanto móveis, garantindo uma experiência de usuário consistente e intuitiva.

2. Arquitetura do Sistema

O backend do PRISMA TAI é construído em Node.js com Express, utilizando TypeScript para a tipagem estática. A arquitetura segue um padrão em camadas, separando as responsabilidades em:

- **Controllers:** Responsáveis por receber as requisições HTTP, validar os dados de entrada e orquestrar a execução das operações, invocando os serviços apropriados.
- **Services:** Contêm a lógica de negócio da aplicação. Eles processam os dados, realizam os cálculos necessários e interagem com os repositórios para a persistência dos dados.
- **Repositories:** Abstraem o acesso ao banco de dados, fornecendo uma interface para consultar e manipular os dados sem expor os detalhes da implementação do banco de dados.
- **Models:** Definem a estrutura das tabelas do banco de dados e seus relacionamentos, utilizando o Sequelize como ORM.
- **Utils:** Módulos auxiliares que fornecem funcionalidades reutilizáveis, como cálculos estatísticos, geração de tokens JWT, formatação de dados e manipulação de arquivos.
- **Middlewares:** Funções que interceptam as requisições antes de chegarem aos controllers, utilizadas para tarefas como autenticação, tratamento de erros e configuração de CORS.

3. Estrutura do Banco de Dados

O sistema utiliza um banco de dados relacional (MySQL) para armazenar todas as informações. O esquema do banco de dados é bem definido e normalizado, garantindo

a integridade e a consistência dos dados. As principais tabelas incluem:

- **Admin:** Armazena as informações dos administradores do sistema.
- **Pesquisador:** Contém os dados dos pesquisadores, incluindo informações de autenticação e status da conta.
- **Pesquisa:** Tabela central que armazena as informações sobre cada pesquisa, como título, descrição, status, tipo e configurações.
- **Blocos_IAT:** Armazena a estrutura dos blocos de um Teste de Associação Implícita, incluindo o tipo, a ordem e as configurações de cada bloco.
- **Respostas:** Registra as respostas de cada participante a uma pesquisa, incluindo os scores calculados (D-score) e metadados.
- **RespostasBloco:** Armazena os detalhes de cada tentativa de resposta de um participante dentro de um bloco, como latência e acerto.
- **Estatísticas:** Guarda as estatísticas gerais de cada sessão de resposta, como média de latência, desvio padrão e total de erros.
- **LinksAcesso:** Tabela para gerenciar os links de acesso únicos para cada pesquisa, controlando o uso e a expiração.
- **Acessos:** Registra cada acesso a um link de pesquisa, armazenando informações como endereço IP e user-agent.
- **LogAcoesAdmin:** Tabela para auditoria, registrando as ações realizadas pelos administradores no sistema.

4. Análise de Requisitos e Status de Implementação

A seguir, uma análise detalhada dos requisitos funcionais (RF) e não funcionais (RNF) definidos no documento, e o status de sua implementação no backend.

Requisitos Funcionais (RF)

ID	Título	Descrição	Status	Observações
RF01	Seleção do tipo de pesquisa	O pesquisador(a) pode escolher entre aplicar pesquisa de associação explícita, implícita ou ambas.	Parcialmente Implementado	A estrutura do banco de dados suporta a diferenciação dos tipos de pesquisa, mas a lógica para criar e gerenciar pesquisas explícitas ainda não está totalmente implementada.
RF02	Geração e compartilhamento	O sistema deve permitir que o	Implementado	O <code>linkAcessoService.ts</code> e o

	de link	pesquisador(a) gere um link único da pesquisa para enviar aos participantes.		<code>linksAcessoController.ts</code> implementam a geração, validação e gerenciamento de links de acesso.
RF03	Fornecer um template	Inicialmente, o sistema deve oferecer um único template, que segue o modelo de Greenwald.	Implementado	A função <code>gerarBlocosPadraoIAT</code> no arquivo <code>constants.ts</code> cria os 7 blocos padrão do IAT, seguindo o modelo de Greenwald.
RF04	Exportação de dados em Excel	O sistema deve permitir que os dados de resposta (como latência, erros, D-score etc.) sejam exportados em formato .xls/.xlsx (Excel).	Implementado	O <code>resultadoService.ts</code> e o utilitário <code>excel.ts</code> implementam a funcionalidade de exportação de dados para Excel.
RF05	Cadastro do pesquisador	O sistema deve permitir o cadastro e autenticação de pesquisadores com segurança (login por email e senha).	Implementado	O <code>authService.ts</code> e <code>authController.ts</code> cuidam da autenticação. O cadastro de pesquisadores é realizado através de seeders, mas uma rota de cadastro para novos pesquisadores não foi implementada.
RF06	Adição de novos templates	O pesquisador(a) pode importar ou criar templates personalizados para reutilização em outras pesquisas.	Não Implementado	Não há funcionalidade implementada que permita a criação ou importação de templates de pesquisa.
RF07	Adicionar blocos à pesquisa de associação implícita	O pesquisador(a) pode adicionar blocos para compor a estrutura da pesquisa de	Implementado	A criação de blocos é gerenciada pelo <code>blocoIATService.ts</code> e <code>blocoIATController.ts</code> .

		associação implícita (IAT).		
RF08	Editar blocos da associação implícita	O pesquisador(a) pode modificar o conteúdo da associação implícita mesmo após salvar.	Implementado	O <code>blocoIATService.ts</code> e <code>blocoIATController.ts</code> permitem a atualização dos blocos.
RF09	Excluir blocos da associação implícita	O pesquisador(a) pode remover permanentemente blocos da associação implícita, mantendo apenas os essenciais.	Implementado	A exclusão de blocos é suportada pelo <code>blocoIATService.ts</code> e <code>blocoIATController.ts</code> .
RF10	Definir critérios para sinalização de dados inválidos	O pesquisador(a) pode definir critérios como latência mínima (ex: < 300ms) ou percentual de erro (ex: > 10%) para que o sistema identifique e destaque essas respostas.	Implementado	O <code>participanteService.ts</code> possui a lógica para sinalizar sessões inválidas com base em critérios de latência e percentual de erros.
RF11	Cálculo automático do D-score	O sistema deve calcular automaticamente o D-score com base no tempo de resposta entre blocos congruentes e incongruentes.	Implementado	O cálculo do D-score é realizado no <code>dscore.ts</code> e integrado no <code>participanteService.ts</code> e <code>iatProcessingService.ts</code> .
RF12	Registro de metadados do teste IAT	O sistema deve armazenar data, hora de início/fim, latências, erros e D-scores (da e db) por participante.	Implementado	As tabelas <code>Respostas</code> e <code>RespostasBloco</code> armazenam esses dados detalhados para cada sessão.
RF13	Cálculo de estatísticas	O sistema deve calcular estatísticas como	Implementado	O <code>estatisticas.ts</code> contém as funções para calcular o

		média de acertos/erros e desvio padrão das latências da associação implícita.		percentual de acertos e o desvio padrão, e o <code>participanteService.ts</code> utiliza essas funções.
RF14	Configuração dos blocos IAT	O pesquisador(a) pode configurar até 7 blocos, os tipos (prática ou teste) e tentativas permitidas.	Implementado	A criação e configuração dos blocos são gerenciadas através do <code>blocoIATService.ts</code> .
RF15	Inclusão de blocos de treino	O sistema deve permitir a inclusão de blocos de treino (1 e 5) que não impactam no cálculo do D-score.	Implementado	O sistema diferencia os tipos de bloco (treino, teste, prática) e o cálculo do D-score utiliza apenas os blocos de teste.
RF16	Uso de mídia e texto nos blocos	O pesquisador(a) pode optar por incluir ou não imagens, palavras ou ambos nos blocos da associação implícita.	Implementado	O middleware <code>upload.ts</code> com o Multer permite o upload de imagens para os blocos.
RF17	Adicionar perguntas à associação explícita	O pesquisador(a) pode adicionar perguntas.	Não Implementado	A lógica para adicionar e gerenciar perguntas de associação explícita não está presente no backend.
RF18	Excluir perguntas da associação explícita	O pesquisador(a) pode remover permanentemente perguntas da associação explícita, mantendo apenas as essenciais.	Não Implementado	Consequência da não implementação do RF17.
RF19	Editar perguntas da associação explícita	O pesquisador(a) pode modificar o conteúdo da associação	Não Implementado	Consequência da não implementação do RF17.

		explícita mesmo após salvar.		
--	--	------------------------------	--	--

Requisitos Não Funcionais (RNF)

ID	Título	Descrição	Status	Observações
RNF01	Sistema Web Responsivo	O sistema deve funcionar em mobile e desktop, com layout adaptável.	Não Aplicável ao Backend	Este é um requisito do frontend. O backend está pronto para servir a qualquer tipo de cliente.
RNF02	Dados anônimos	O sistema deve garantir anonimato através de IDs únicos não rastreáveis e exclusão de metadados identificáveis.	Implementado	O sistema utiliza UUIDs para sessões e links, garantindo o anonimato dos participantes.
RNF03	Interface intuitiva	O sistema deve apresentar uma interface intuitiva, com navegação simples e acessível.	Não Aplicável ao Backend	Requisito do frontend.
RNF04	Segurança dos dados armazenados	O sistema deve garantir a segurança dos dados armazenados, implementando criptografia, controle de acesso e backups periódicos.	Implementado	A segurança é tratada com hashing de senhas (bcrypt) e autenticação baseada em JWT. A segurança do banco de dados (backups, etc.) depende da infraestrutura de implantação.
RNF05	Armazenamento de dados brutos	O sistema deve armazenar dados brutos (ex.: latências por tentativa) para reprocessamento.	Implementado	A tabela RespostasBloco armazena os dados brutos de cada tentativa, permitindo o reprocessamento.
RNF06	Precisão de tempo	O sistema deve registrar a latência com precisão de milissegundos.	Implementado	A latência é armazenada em milissegundos na tabela RespostasBloco .

RNF07	Desempenho	O sistema deve carregar rapidamente, processar estatísticas em tempo real e manter seu desempenho mesmo com alta concorrência de participantes.	Parcialmente Implementado	A arquitetura é escalável, mas testes de carga e otimizações de performance não foram realizados. O processamento das estatísticas é feito ao final de cada sessão.
RNF08	Banco de Dados Relacional	O sistema deve utilizar um banco de dados SQL para armazenar os dados.	Implementado	O sistema utiliza o Sequelize com um banco de dados MySQL.

5. Funcionalidades Implementadas

- **Autenticação e Autorização:** Sistema de login seguro para pesquisadores e administradores com JWT.
- **Gerenciamento de Pesquisas:** Criação, edição, exclusão, publicação e despublicação de pesquisas.
- **Gerenciamento de Blocos IAT:** Criação, edição e exclusão de blocos para as pesquisas IAT.
- **Geração de Links de Acesso:** Criação de links únicos para as pesquisas, com controle de validade e número de usos.
- **Coleta de Respostas:** Endpoints para receber e armazenar as respostas dos participantes em tempo real.
- **Processamento e Análise de Dados:** Cálculo automático do D-score e outras estatísticas relevantes ao final de cada sessão.
- **Exportação de Dados:** Funcionalidade para exportar os resultados das pesquisas em formato Excel.
- **Estrutura de Banco de Dados Robusta:** Um esquema de banco de dados bem definido e modelado com Sequelize.
- **Tratamento de Erros e Logging:** Middlewares para tratamento de erros e um sistema de logging com Winston.

6. Funcionalidades Faltantes e Pontos de Melhoria

- **Módulo de Pesquisa Explícita:** A funcionalidade mais significativa que falta é o suporte completo para pesquisas explícitas. Isso incluiria a criação de perguntas,

definição de tipos de resposta (múltipla escolha, escala Likert, etc.) e a coleta e análise desses dados.

- **Cadastro de Novos Pesquisadores:** Atualmente, os pesquisadores são cadastrados via seed. É necessário implementar uma rota de cadastro para que novos pesquisadores possam se registrar na plataforma.
- **Gerenciamento de Templates:** A funcionalidade de criar e reutilizar templates de pesquisa não foi implementada.
- **Interface Administrativa:** Faltam endpoints para que os administradores possam gerenciar pesquisadores (aprovar, desativar) e pesquisas (revisar, aprovar). A estrutura do banco de dados já suporta essas funcionalidades.
- **Testes Automatizados:** Não há testes unitários ou de integração, o que é crucial para garantir a qualidade e a manutenibilidade do código.
- **Documentação da API:** Embora o código esteja bem estruturado, falta uma documentação formal da API (como Swagger/OpenAPI) para facilitar o desenvolvimento do frontend e a integração com outros sistemas.
- **Otimização de Performance e Escalabilidade:** São necessários testes de carga para identificar gargalos e otimizar as consultas ao banco de dados e a lógica de processamento para garantir o bom desempenho com um grande número de usuários simultâneos.

7. Representação visual



